

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

# SE 423: Introduction to Mechatronics

## Lecture 11: PID Control

Marius Juston

Wednesday, February 25th, 2026

# Reminders

**Office Hours:**

- **Monday:** 4-6 PM, Neel
- **Tuesday:** 11-1 PM, Lakshmi
- **Tuesday:** 1-3 PM, Samuel
- **Tuesday:** 2-5 PM, Dan & Marius

**Lab:** Starting Lab 4 this week. This a 2-week lab!

**Homeworks:**

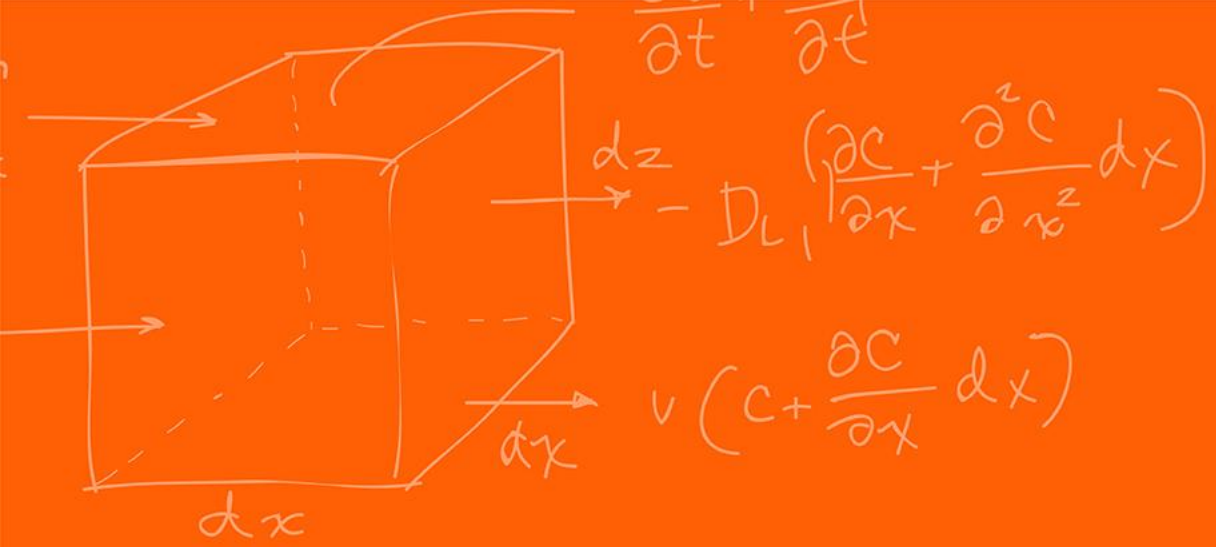
- Answers and code submissions for homework 2 are due **February 26, 9AM**
- Check-offs for homework 3 Ex 1, 2, 3 & 4 are due **March 10, 5PM (The end of office hours)**
- Check-offs for homework 3 Ex 5 are due **March 24, 5PM (The end of office hours)**
- Answers and code submissions for homework 3 are due **March 25, 9AM**

**LabVIEW:**

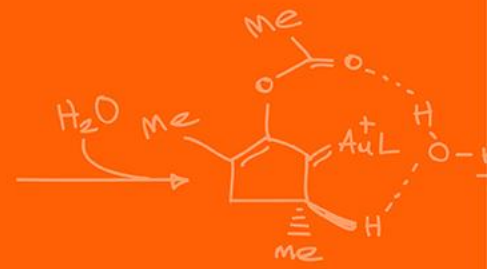
- All check-offs for LabVIEW 2 are due **February 26, 5PM**

# Questions? (1/2)

Homework, Lab, LabVIEW questions?

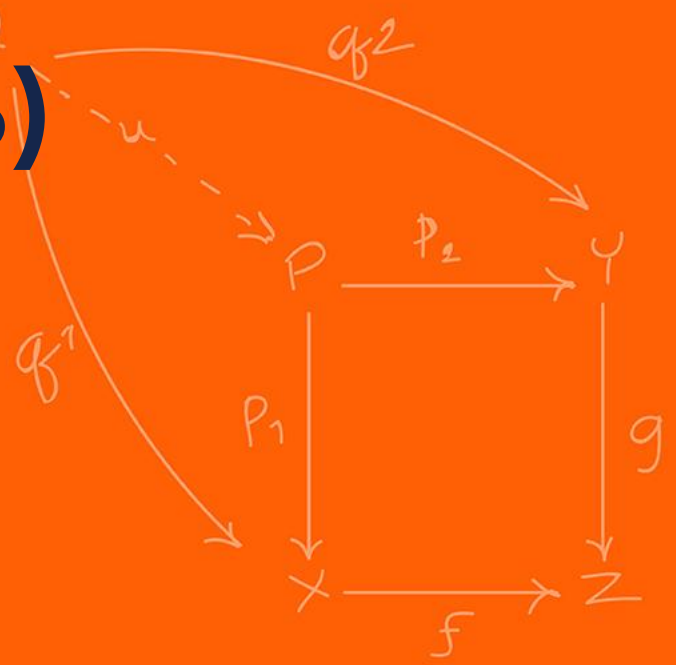








# PID Controller (1/18)



# PID Controller (2/18)

There are 2 types of controllers:

- **Open-Loop** systems: Directly dictates the output.

**Problems?**
Any deviation / noise will cause you to not reach your target.

- **Closed-Loop** systems: Utilizes feedback to compute an error metric, dynamically correcting the control effort

# PID Controller (3/18)

We know it, we love it, it’s the classic PID Controller!

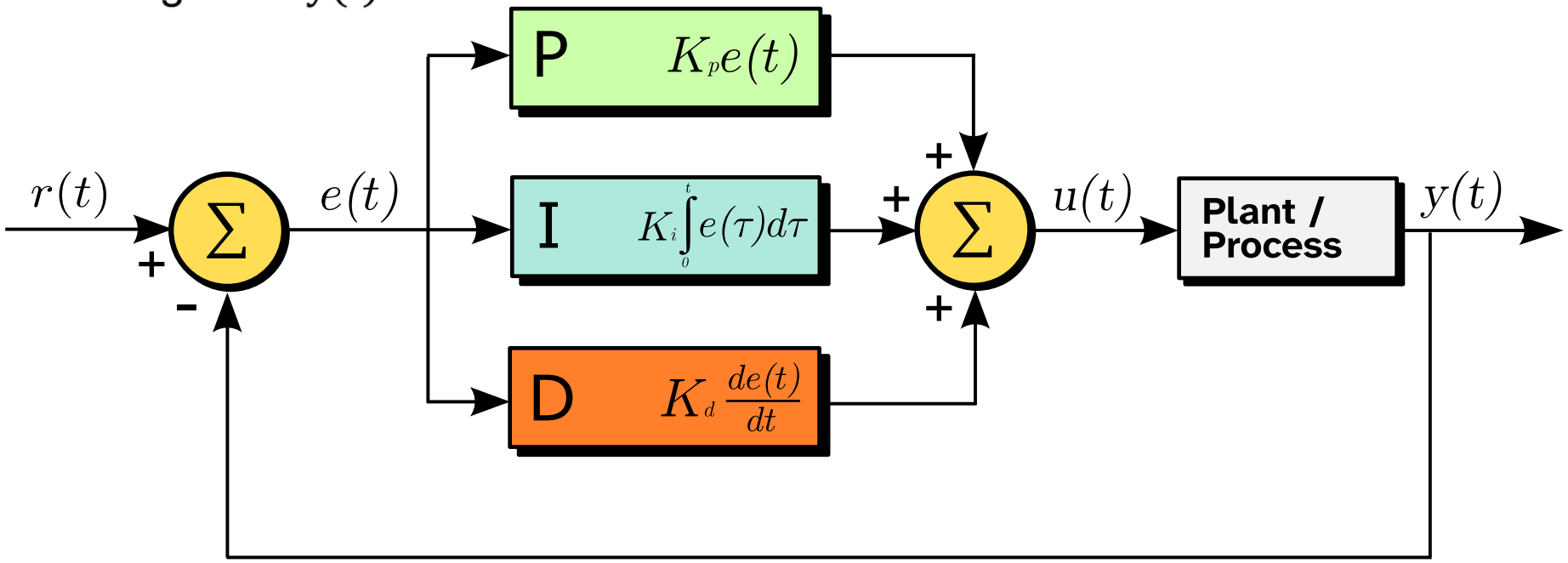
The proportional–integral–derivative (PID) controller  is a closed-loop controller.

It is extremely common due to its simplicity, both in the tuning and in the implementation ( you will see how simple it is when you implement it in the code)

# PID Controller (4/18)

The PID controller is split into 3 parts, as its name states. While each of the terms contribute to improving the output, it is not necessary to use all of them at once.

As the PID controller is a closed-loop controller the input is an error signal $e(t) = r(t) - y(t)$, which represents the reference "what you want to be doing" $r(t)$ and the currently measured input "what you are doing now" $y(t)$




https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller

# PID Controller (5/18)

We are going to watch some really good MATLAB videos (I highly recommend looking at them some more!)

What is a PID Controller?

There are 2 main problems with the basic implementation of the PID loop in a real-world example. Any ideas?

- Integrator Windup
- Derivative noise

For the derivative term we decide that it is not worth using and the robot will follow the path well enough. It is usually more recommended to use if you have a sensor the derivative directly!

# PID Controller (6/18)

Term **P** is the proportional term. It is simplify defined as,

$$u(t) = K_P e(t),$$

for example, if the error is large, the output control effort $u(t)$ will be large and vice-versa.

This is the most fundamental part of the PID loop, you should at least always use this term.

# PID Controller (7/18)

Term I accounts for past values of the error and integrates them over time to produce the I term

$$u(t) = K_I \int_0^t e(t),$$

For example, if there is a residual error after the application of proportional control, the integral terms eliminates the residual error by adding some additional control effort.

When the error is eliminated, the integral term will cease to grow.

It is important to bound this component such that it does not explode and make it difficult to control the system. I.e. integrator windup.

# PID Controller (8/18)

- Term **D** accounts for future values of the error based on the current rate of change of the error.

$$u(t) = K_D \frac{de(t)}{dt},$$

In the real world, sensors are noisy and so collecting the derivative term can be difficult without an additional filter to it.

# PID Controller (9/18)

The full form is thus,

$$u(t) = K_p e(t) + K_I \int_0^t e(t) + K_D \frac{de(t)}{dt},$$

Simple as that!

# PID Controller (10/18)

```c
#define MAX_EFFORT 10.0f

struct PID {
    float Kp;
    float Ki;
    float integral;
};

float saturate(float input, float satVal){
    input = (input > satVal)? satVal :
input;
    input = (input < -satVal)? -satVal :
input;

    return input;
}

int sign(float x){
    if (x > 0.0f) return 1;
    if (x < 0.0f) return -1;
    return 0;
}
```

```c
float PIControl(float reference, float target, float dt, struct PID *pidState){
    if (dt <= 0.0f) return 0.0f;

    float error = reference - target;

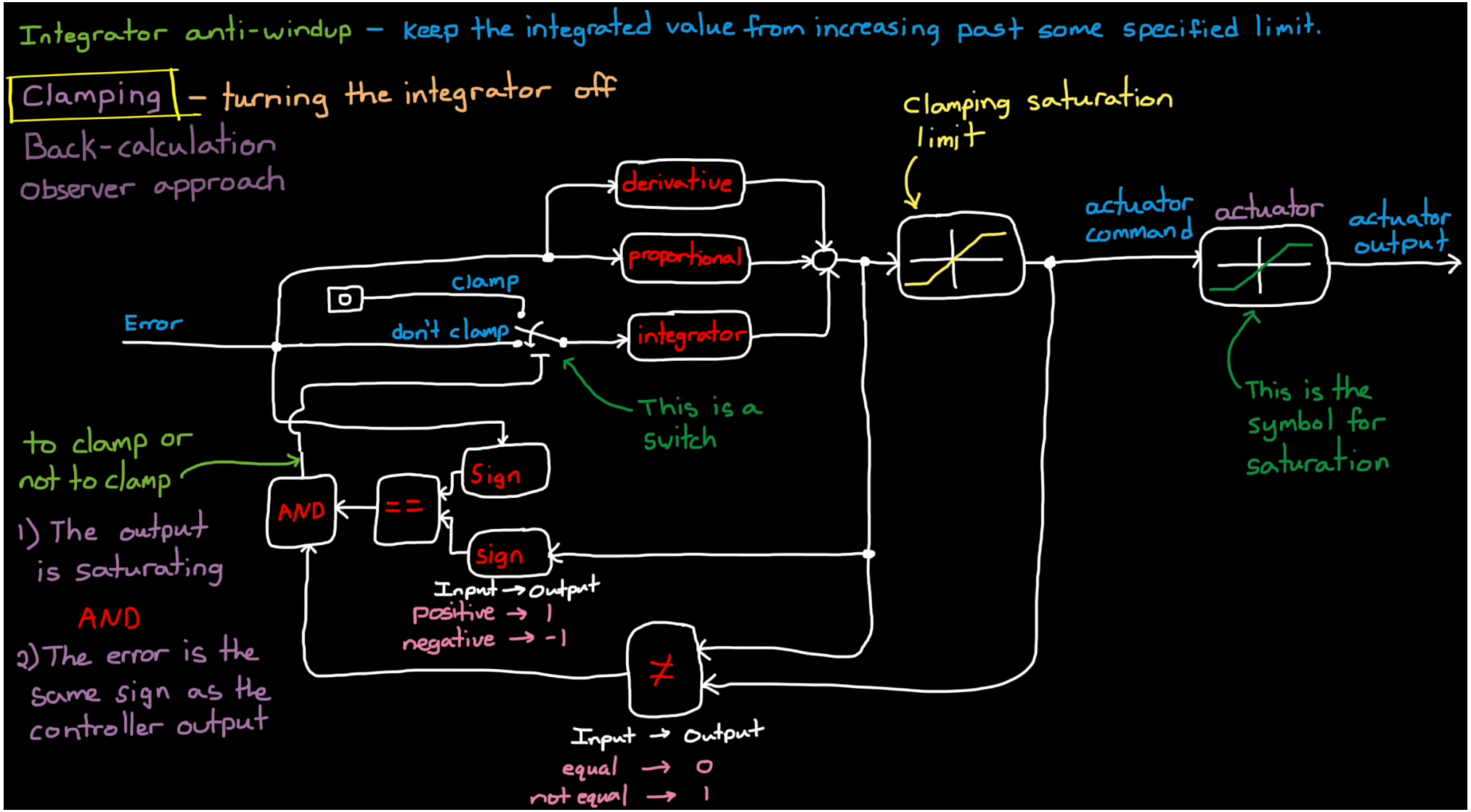
    // Compute integral
    float integral = pidState->integral + error * dt;
    // Compute PI using tentative integral
    float effort = pidState->Kp * error +
                   pidState->Ki * integral;

    // Saturate control input
    float effortSat = saturate(effort, MAX_EFFORT);

    // Anti-windup condition
    // The output is saturating
    // The error is the same sign as the controller output
    if(!(effort != effortSat && sign(effort) == sign(error))){
        pidState->integral = integral;
    }

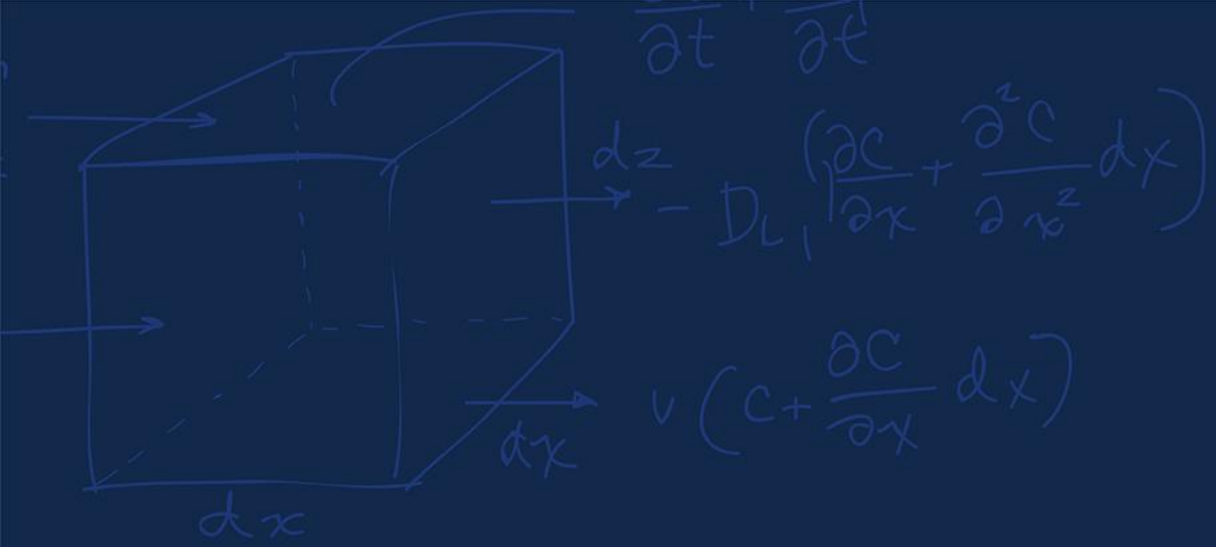
    return effortSat;
}
```

# PID Controller (11/18)

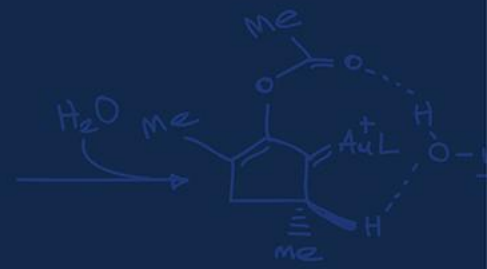



https://www.mathworks.com/videos/understanding-pid-control-part-2-expanding-beyond-a-simple-integral-1528310418260.html

# Tuning



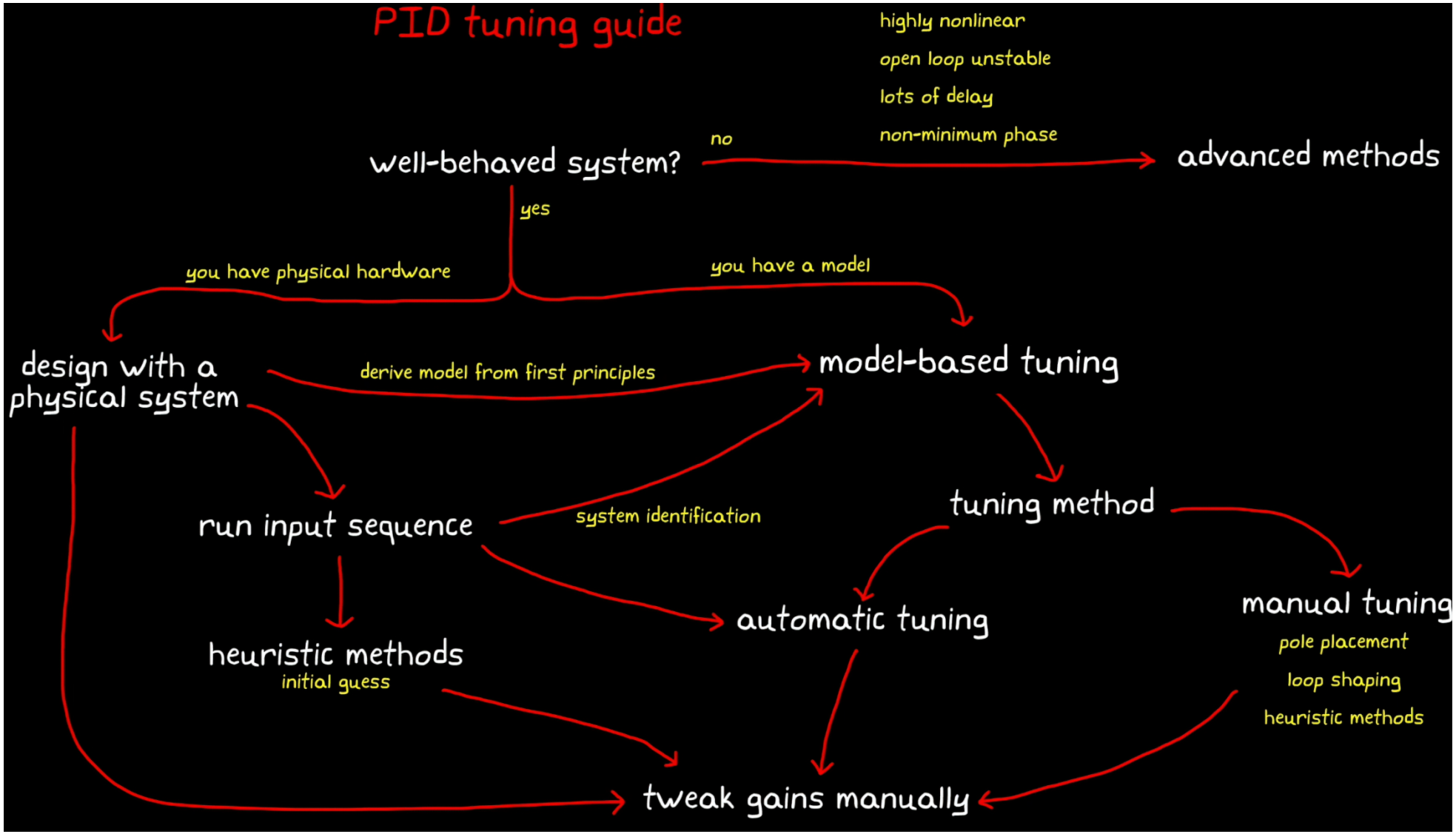








# PID Controller (12/18)




https://www.mathworks.com/videos/understanding-pid-control-part-4-a-pid-tuning-guide-1531124725727.html

# PID Controller (13/18)

In this class we will only be doing manual tuning, it is just intuition based but given that there are only 3 gains, it is fast to find appropriate constants!

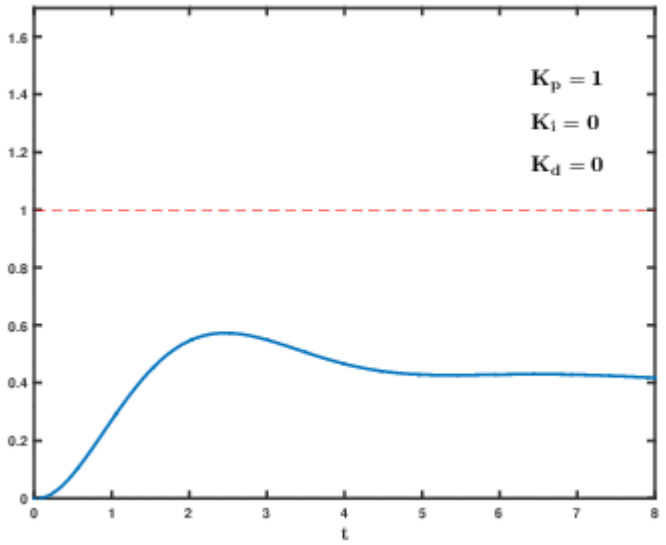
The manual method for tuning is simple:

1. Set all gains to 0 and start $K_p$ at 1
2. Keep increasing $K_P$ until the output of the system causes it to oscillate
3. Then set $K_P$ approximately half that value
4. Then increase $K_I$ until any offset is corrected
5. Then, optionally, increase $K_D$ to remove overshoot

Usually, $K_P$ has the highest magnitude, $K_I$ next and then $K_D$.

# PID Controller (14/18)

| Parameter | Rise time | Overshoot | Settling time | Steady-state error | Stability |
|---|---|---|---|---|---|
| $K_p$ | Decrease | Increase | Small change | Decrease | Degrade |
| $K_i$ | Decrease | Increase | Increase | Eliminate | Degrade |
| $K_d$ | Minor change | Decrease | Decrease | No effect in theory | Improve if $K_d$ small |



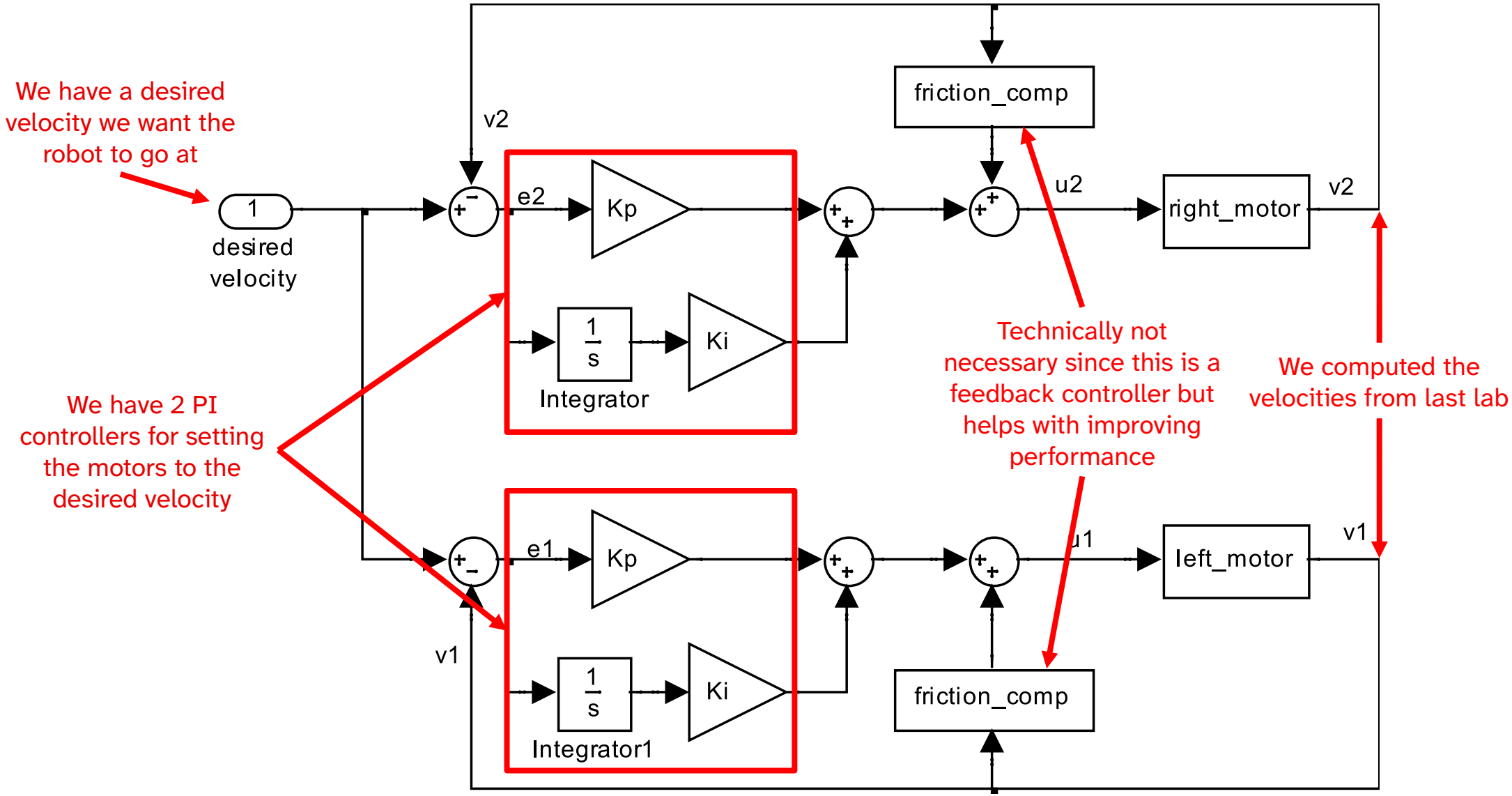

https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller#Manual_tuning

# PID Controller (15/18)

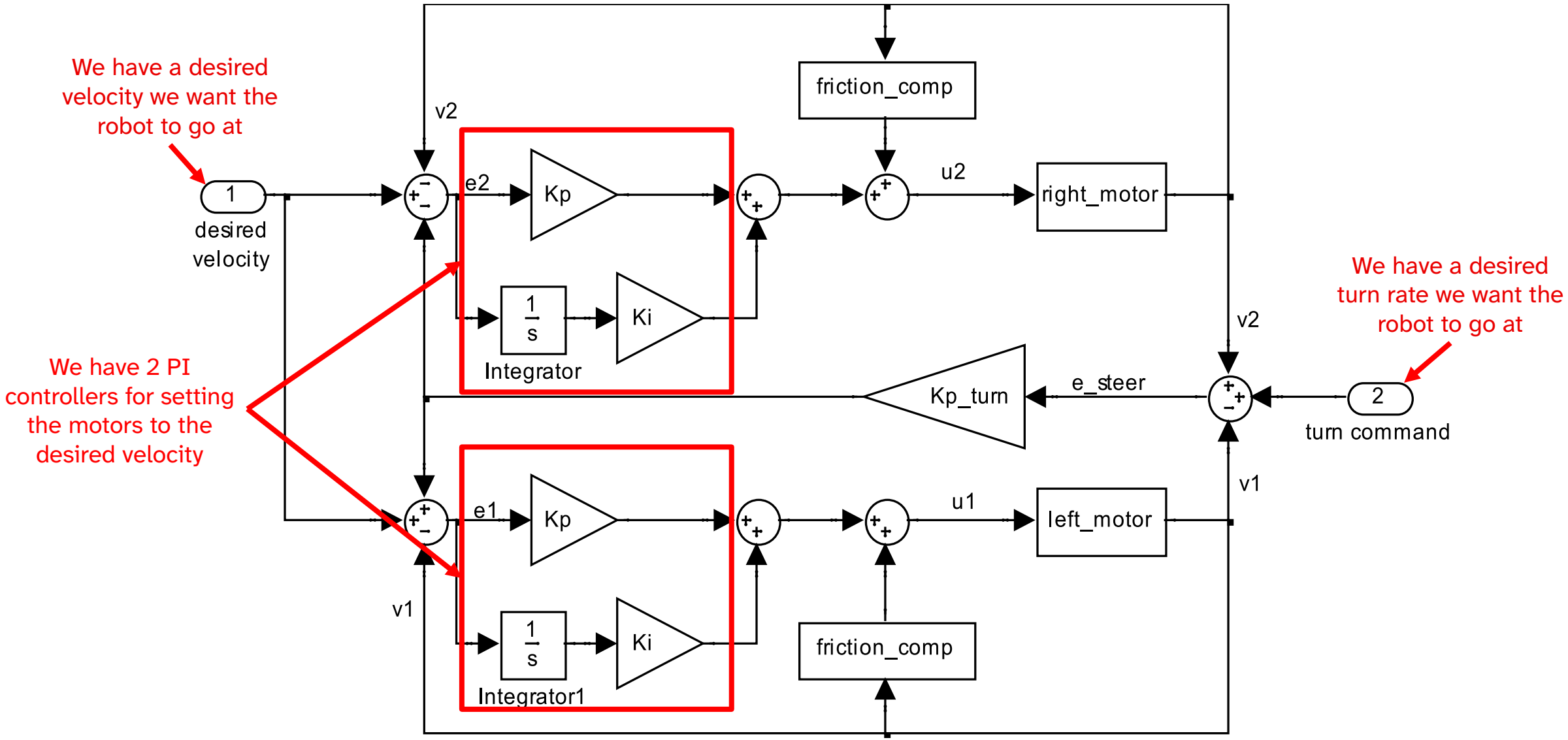
We can try out the effect of gains here!

https://www.luisllamas.es/en/pid-controller-simulator/

# PID Controller – Lab 5



# PID Controller (16/18)



# PID Controller (17/18)

We have 3 error terms,

First, we have the steering error $e_{steer}(t)$

$$e_{steer}(t) = \omega_r(t) - ( v_l(t) - v_r(t) )$$

If we tell the robot to turn right ($\omega_r(t) > 0$), we expect the right wheel to move faster than the left wheel.

This is because in a differential drive system, the yaw rate is proportional to the difference between the right and left velocities $\omega \propto v_r - v_l$

# PID Controller (18/18)

Now we have the **Left Motor error** $e_l(t)$, receives positive feedback from the turning loop

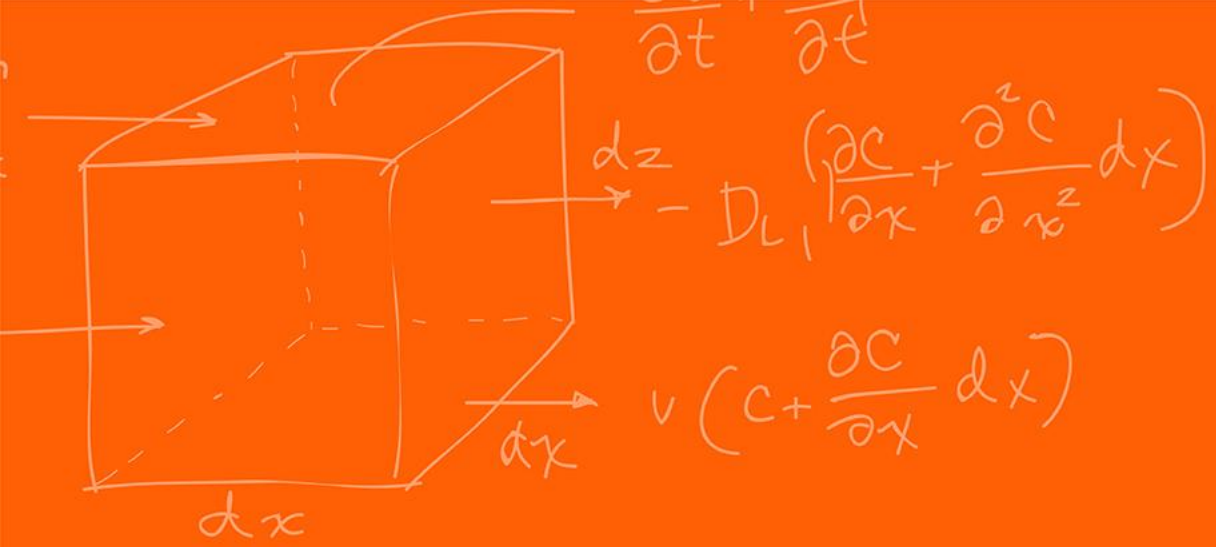
$$e_l(t) = v_r(t) - v_l(t) + K_{steer} e_{steer}(t)$$

Now we have the **Right Motor error** $e_r(t)$, receives negative feedback from the turning loop

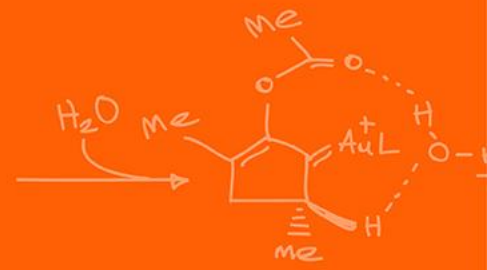
$$e_r(t) = v_r(t) - v_l(t) - K_{steer} e_{steer}(t)$$

This cross couples the control loops for the steering and the velocity control. Such that you have a feedback loop for making the robot steer. And then a separate control loop to reach the robot's target individual motor velocity.

# Questions? (2/2)









# The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

